





UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

Sistemas Distribuidos

# REPORTE LABORATORIO 9

GRUPO: 02

ALUMNOS:

MEDRANO SOLANO ENRIQUE - 320152841

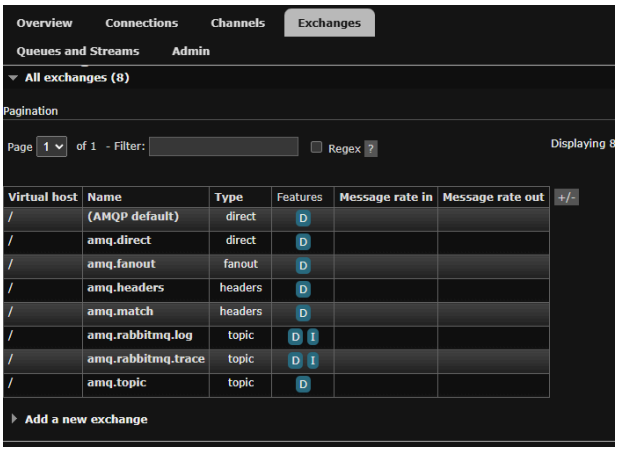
FECHA DE ENTREGA: 24 de octubre de 2026

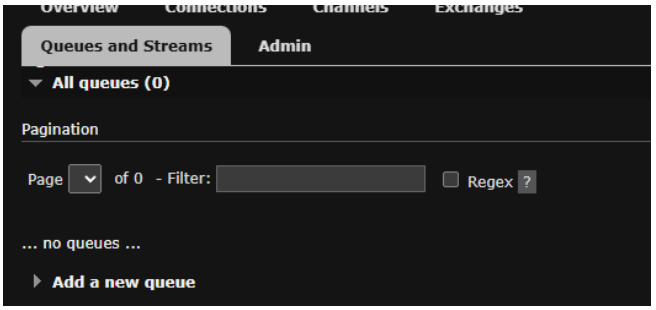
PROFESOR: Ing. Oscar Manuel Ruiz Hurtado

**Desarrollo.**

1. **El problema.**

Se verifica que en RabbitMQ el panel en los parámetros dichos vengan como se espera.




Ahora se crean las cuentas y el monto inicial de estas mismas, en este caso todo salió en orden, con el mensaje de 201 created.

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   132    0    66  100    66    116    116 --:--:-- --:--:-- --:--:--   233HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 9ce0902660b3
Content-Type: application/json
Date: Sat, 24 Oct 2026 15:41:49 GMT

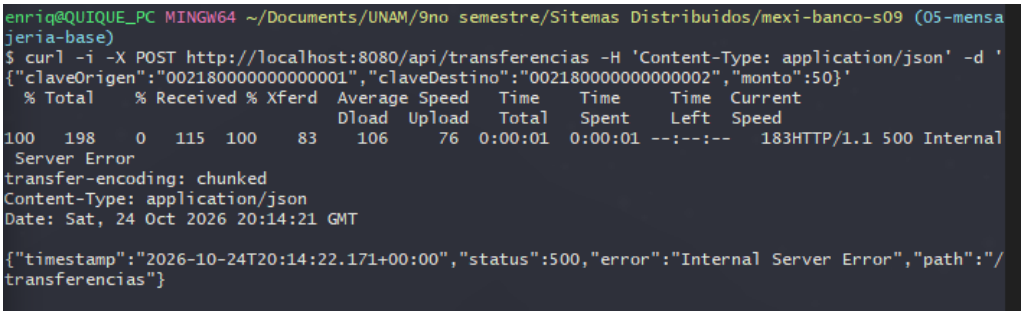
{"id":3,"clabe":"002180000000000001","titular":"Ana","saldo":1000}

enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   132    0    66  100    66    238    238 --:--:-- --:--:-- --:--:--   478HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 2778f28e7708
Content-Type: application/json
Date: Sat, 24 Oct 2026 15:42:08 GMT

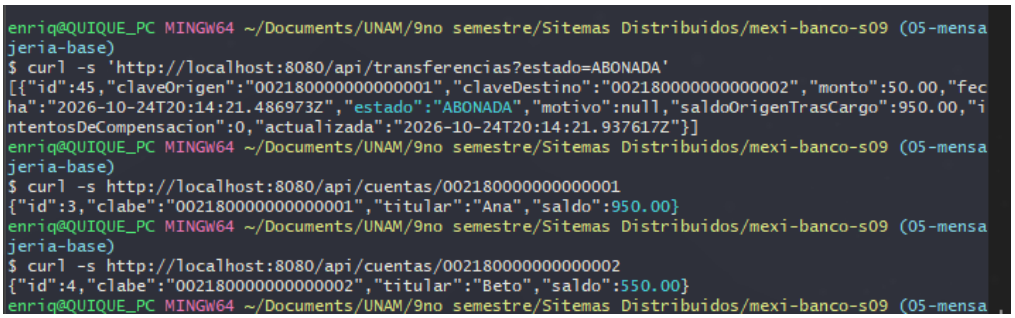
{"id":4,"clabe":"002180000000000002","titular":"Beto","saldo":500}
```

Ahora lo que se realiza es el apagado de las notificaciones y hacer la transferencia.

```shell
docker compose stop notificacion
#Se debería mostrar algo así
 Container mexi-banco-v06-notificacion-1 Stopping
 Container mexi-banco-v06-notificacion-1 Stopped
```



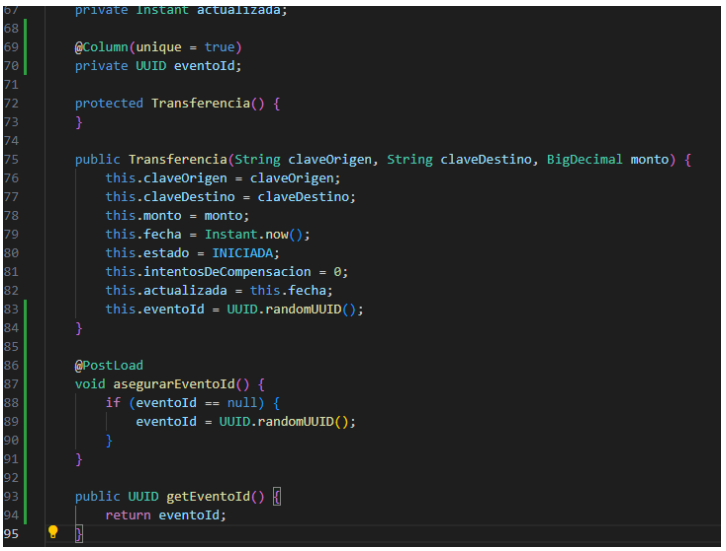

Como se observa, se muestra que ocurrió un error, sobre el servicio no disponible.




El cargo, el abono y los asientos ya ocurrieron. ¿Por qué el cliente recibe un error?

Aquí ya se puede observar que realmente el estado ABONADA, pero la saga esperaba una respuesta del HTTP síncrona con el servicio de notificaciones. Al estar apagado, la espera lanzó una excepción, de manera que la ejecución aborto antes de otra instrucción. El cargo y abono se ven reflejados.

2. **Publicar.**



```
    private Instant actualizada;

    @Column(unique = true)
    private UUID eventoId;

    protected Transferencia() {
    }

    public Transferencia(String claveOrigen, String claveDestino, BigDecimal monto) {
        this.claveOrigen = claveOrigen;
        this.claveDestino = claveDestino;
        this.monto = monto;
        this.fecha = Instant.now();
        this.estado = INICIADA;
        this.intentosDeCompensacion = 0;
        this.actualizada = this.fecha;
        this.eventoId = UUID.randomUUID();
    }

    @PostLoad
    void asegurarEventoId() {
        if (eventoId == null) {
            eventoId = UUID.randomUUID();
        }
    }

    public UUID getEventoId() {
        return eventoId;
    }
}
```



```
    }
        You, 10 minutes ago • Uncommitted changes
    private void avisar(EventoTransferencia evento) {
    try {
        eventos.publicar(evento);
        log.info("Transferencia {}: evento {} publicado ({})", evento.transferenciaId(),
                evento.tipo(), evento.eventoId());
    } catch (RuntimeException sinPublicar) {
        log.error("Transferencia {}: quedo terminada pero el evento {} {} NO se publico: {}",
                evento.transferenciaId(), evento.tipo(), evento.eventoId(),
                sinPublicar.getMessage());
    }
}
}
```



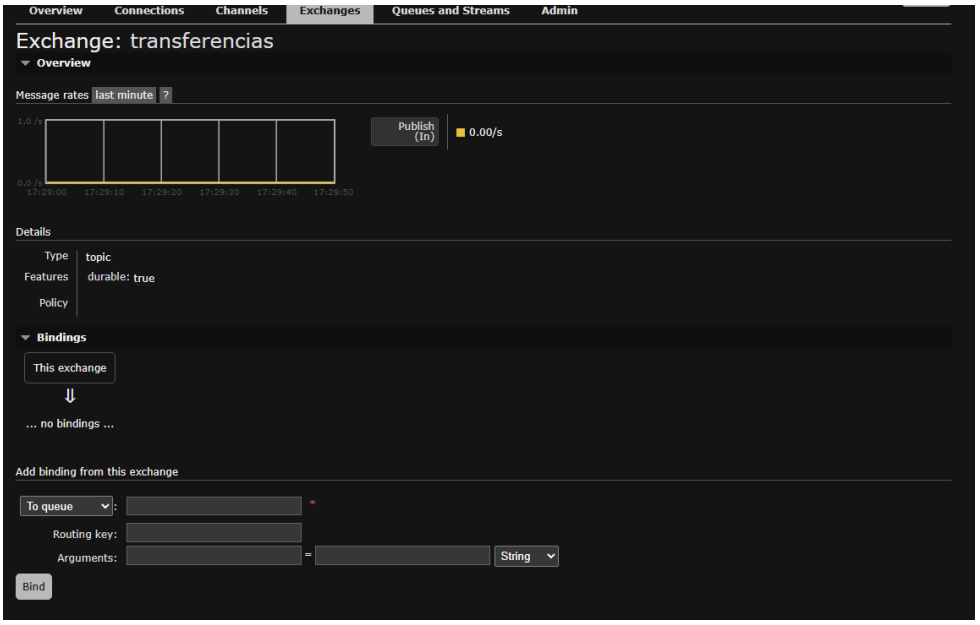
```
public void publicar(EventoTransferencia evento) {
    String id = evento.eventoId().toString();
    CorrelationData correlacion = new CorrelationData(id);    Null type safety: The expression of type 'String'
    try {
        rabbit.convertAndSend(ConfiguracionMensajeria.EXCHANGE, evento.routingKey(), evento, mensaje -> {    Nul
            mensaje.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT); // a disco
            mensaje.getMessageProperties().setMessageId(id); // se ve en el panel sin abrir el JSON
            return mensaje;
        }, correlacion);
    } catch (RuntimeException sinConexion) {
        throw new EventoNoPublicadoException(
                "No se pudo mandar el evento " + id + " al broker: " + sinConexion.getMessage(), sinConexion);
    }
    CorrelationData.Confirm confirmacion;
    try {
        confirmacion = correlacion.getFuture().get(esperaConfirmacionMs, TimeUnit.MILLISECONDS);
    } catch (TimeoutException | ExecutionException sinRespuesta) {
        throw new EventoNoPublicadoException("El broker no confirmo el evento " + id + ": " + sinRespuesta,
                sinRespuesta);
    } catch (InterruptedException interrumpido) {
        Thread.currentThread().interrupt();
        throw new EventoNoPublicadoException("Se interrumpio la espera del evento " + id, interrumpido);
    }
    if (!confirmacion.isAck()) {
        throw new EventoNoPublicadoException(
                "El broker rechazo el evento " + id + " (nack): " + confirmacion.getReason());
    }
    ReturnedMessage regresado = correlacion.getReturned(); // lo regresó por mandatory
    if (regresado != null) {
        throw new EventoNoPublicadoException("Ninguna cola recibio el evento " + id + " con la routing key "
                + regresado.getRoutingKey() + " (" + regresado.getReplyText() + ")");
    }
}
}
```

En lugar de usar un cliente HTTP para llamar directamente a otro micorservicio, el servicio de **transferencia** ahora crea un mensaje JSON, documentando el evento cuando ya pasó, y lo deposita en un buzón de RabbitMQ en la parte de **exchange.**

Además se garantiza que el broker recibió el mensaje de manera exitosa y toma la responsabilidad sobre él devolviendo una confirmación. No garantiza que exista una cola enlazada ni que el consumidor haya procesado la imagen, solo se asegura la persistencia en el broker al momento del salto de red.

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '
{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   405    0   322  100    83    426    109 --:--:-- --:--:-- --:--:--   535HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 23:27:30 GMT

{"id":46,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":
"2026-10-24T23:27:30.189069469Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,
"intentosDeCompensacion":0,"actualizada":"2026-10-24T23:27:30.662340837Z","eventoId":"c5f6f955-c9d7
-4fc5-89b1-df863279fcae"}
```




Activaste los publisher confirms. ¿Qué te garantiza la confirmación? ¿Y qué no te garantiza? Piensa en dos casos: que notificación esté apagado, y que el proceso de transferencia muera justo entre guardar(t) y avisar(...).

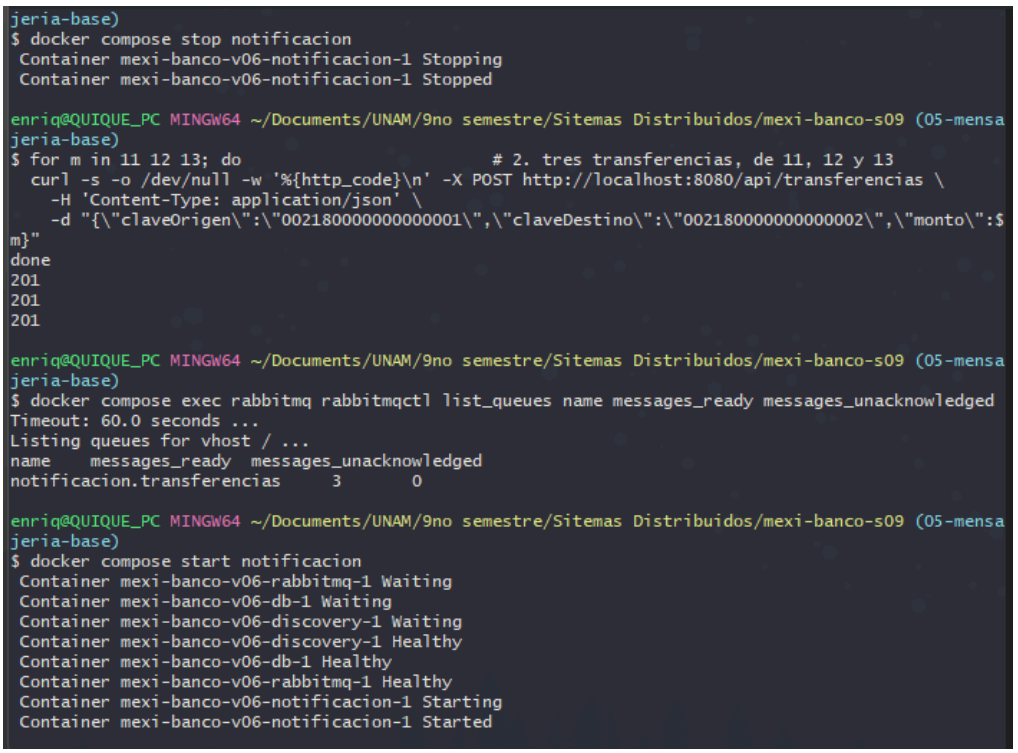
En este caso nos garantiza que RabbitMQ reciba el mensaje correctamente, lo guardó y tomó responsabilidad sobre él, devolviendo un recibo al servicio que lo publica.

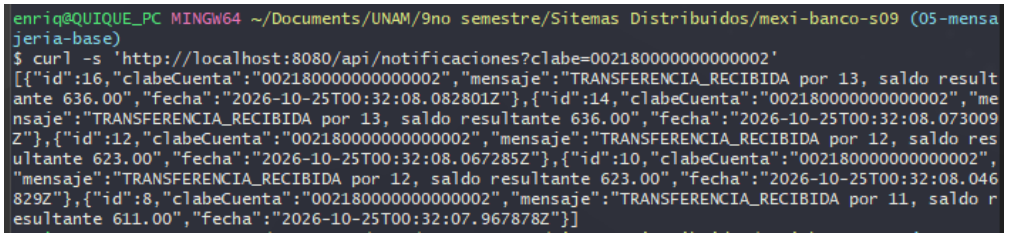
Pero aún así no garantiza que el mensaje haya sido entregado a un consumidor, ni garantiza que exista una cola enlazada para retener el mensaje. En el caso en

donde la notificación esté apagada, RabbitMQ recibirá el mensaje y se confirma su recepción.

En dado caso que el proceso de transferencia muere entre guardar(t) y avisar(), la base de datos registrará la transferencia como completa, pero el evento nunca será publicado en RabbitMQ porque el proceso colapsó antes de ser publicado. El publisher confirm no sirve de nada porque el broker jamás recibió la orden inicial, creando una inconsistencia.

3. **Desacople en tiempo.**




Como se observa en las capturas, esto demuestra el desacople en tiempo: el productor (transferencia) y el consumidor (notificacion) ya no necesitan estar vivos simultáneamente para que el sistema funcione. A pesar de que el servicio de notificaciones estaba apagado, las transferencias se completaron con éxito y los eventos esperaron de forma segura en la cola durable de RabbitMQ. Al reiniciar el

consumidor, este procesó inmediatamente los mensajes acumulados y generó los avisos correspondientes en las bandejas, garantizando que no se perdiera ninguna alerta.

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '
{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150}'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   576    0   492  100    84   1037    177 --:--:-- --:--:-- --:--:--  1215HTTP/1.1 502 Bad Gate
way
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 25 Oct 2026 00:41:49 GMT

{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar
 a la CLABE 002180000000000002 y la devolucion de 150 a la CLABE 002180000000000001 tampoco pudo ha
cerse todavia. El dinero salio de origen; el banco reintentara la devolucion automaticamente.","ins
tance":"/transferencias","transferencia":50,"estado":"COMPENSACION_PENDIENTE","motivo":"El abono a
la CLABE 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=002180000000000002"}
```

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":15,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resulta
nte 864.00","fecha":"2026-10-25T00:32:08.082756Z"},{"id":13,"clabeCuenta":"002180000000000001","men
saje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fecha":"2026-10-25T00:32:08.072965Z"
},{"id":11,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo result
ante 877.00","fecha":"2026-10-25T00:32:08.067241Z"},{"id":9,"clabeCuenta":"002180000000000001","men
saje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00","fecha":"2026-10-25T00:32:08.046747Z"
},{"id":7,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resulta
nte 889.00","fecha":"2026-10-25T00:32:07.967790Z"}]
```

Si el ack va después de guardar: Si el servicio notificacion muere justo después de guardar en la base de datos pero antes de enviar el ack a RabbitMQ, el aviso se repite. Como el broker nunca recibió la confirmación, asume que el procesamiento falló, devuelve el mensaje a la cola y lo vuelve a entregar.

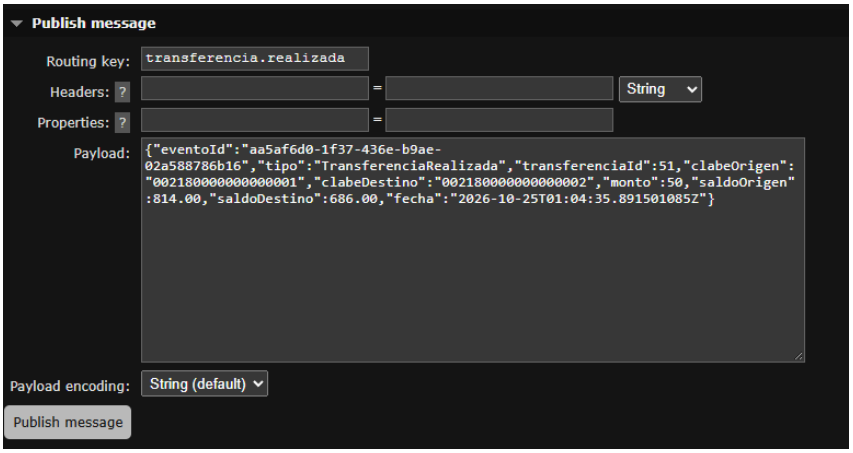
Si el ack va antes de guardar: Si el servicio notificacion muere justo después de enviar el ack a RabbitMQ pero antes de guardarlo en la base de datos, el aviso se pierde para siempre. RabbitMQ elimina el mensaje de la cola creyendo que fue procesado con éxito, y como la base de datos abortó la transacción, el cliente jamás recibirá su notificación.

4. **El duplicado.**

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
jeria-base)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '
{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   405    0   322  100    83   1713    441 --:--:-- --:--:-- --:--:--  2165HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 25 Oct 2026 01:04:35 GMT

{"id":51,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":
"2026-10-25T01:04:35.821714007Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":814.00,
"intentosDeCompensacion":0,"actualizada":"2026-10-25T01:04:35.891501085Z","eventoId":"aa5af6d0-1f37
-436e-b9ae-02a588786b16"}

enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
```




Publicación del segundo mensaje y ahora aparecen dos en Ready en la parte de Queues.

| Virtual host | Name | Type | Features | State | Ready | Unacked | Total | incoming | deliver / get | ack |
|---|---|---|---|---|---|---|---|---|---|---|
| / | notificacion.transferencias | classic | D Args | running | 2 | 0 | 2 | 0.00/s | 0.00/s | 0.00/s |

```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":20,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 50, saldo resultante 686.00","fecha":"2026-10-25T01:09:04.821221Z"},{"id":18,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 50, saldo resultante 686.00","fecha":"2026-10-25T01:09:04.741655Z"},{"id":16,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-25T00:32:08.082801Z"},{"id":14,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-25T00:32:08.073009Z"},{"id":12,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-25T00:32:08.067285Z"},{"id":10,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-25T00:32:08.046829Z"},{"id":8,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-25T00:32:07.967878Z"}]
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensa
```

Se muestran los avisos repetidos, en donde se tendrá el mismo monto y saldos, registrados dos veces.

| Overview | | | | | Messages | | | Message rates | | | +/- |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Virtual host | Name | Type | Features | State | Ready | Unacked | Total | incoming | deliver / get | ack | |
| / | notificacion.transferencias | classic | D Args | running | 0 | 0 | 0 | 0.00/s | 0.00/s | 0.00/s | |

Add a new queue

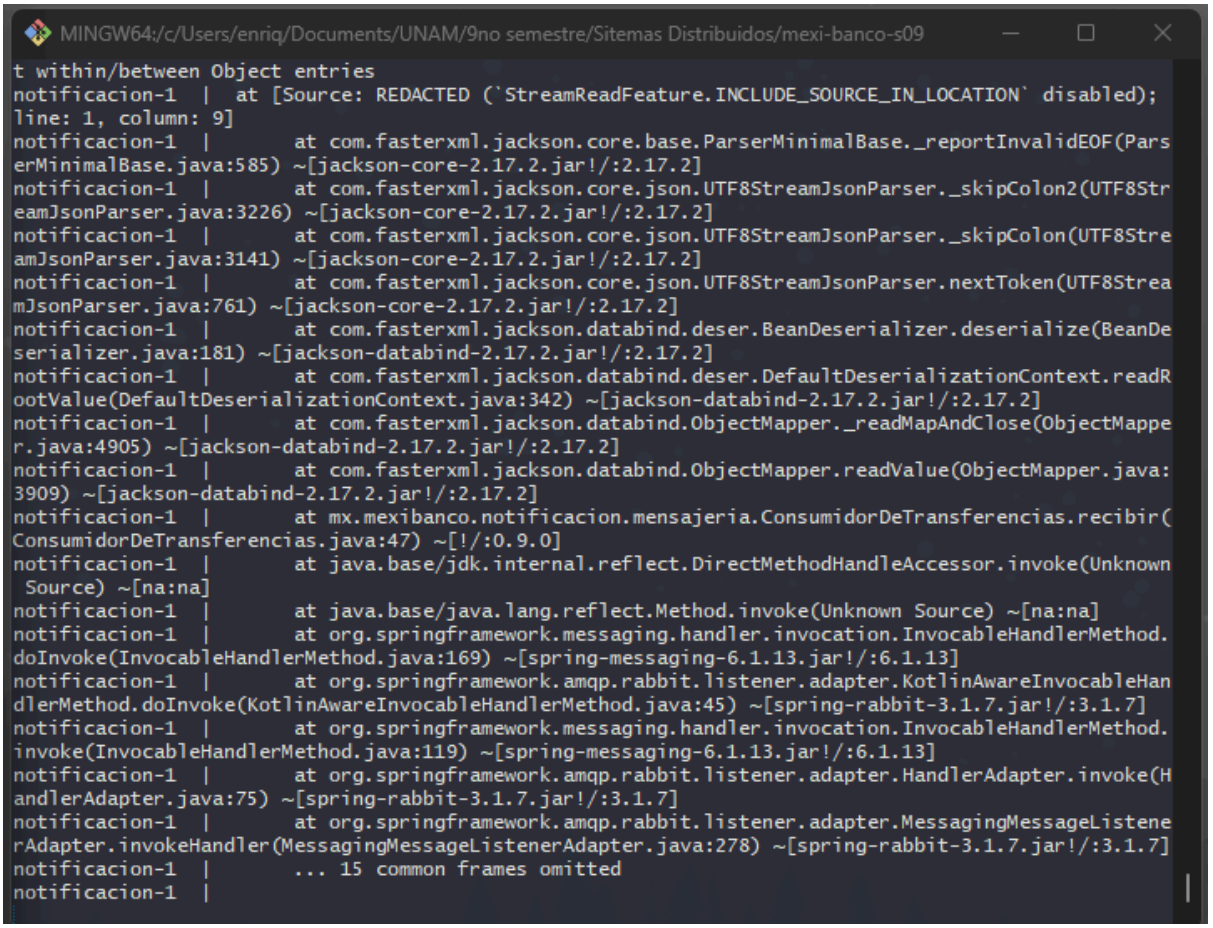
```
enriq@QUIQUE_PC MINGW64 ~/Documents/UNAM/9no semestre/Sitemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
{"timestamp":"2026-10-25T01:23:47.417+00:00","path":"/api/notificaciones","status":503,"error":"Service Unavailable","requestId":"c2babf27-7526"}
```

Ahora al hacer el arreglo, ya no hay mensajes repetidos.

¿Por qué en la misma transacción? Si algo falla después de la marca, el rollback se lleva marca y avisos juntos, y la siguiente entrega lo intenta desde cero.

Para garantizar un comportamiento total. Si guardas la marca de procesado en una transacción diferente, y luego la creación de los avisos falla, el evento quedará marcado como "procesado". Cuando RabbitMQ lo reentregue, el sistema ignorará el mensaje por culpa de esa marca y los avisos se perderán para siempre. Al estar en la misma transacción, si algo falla, se borra la marca y los avisos juntos, permitiendo un reintento limpio.

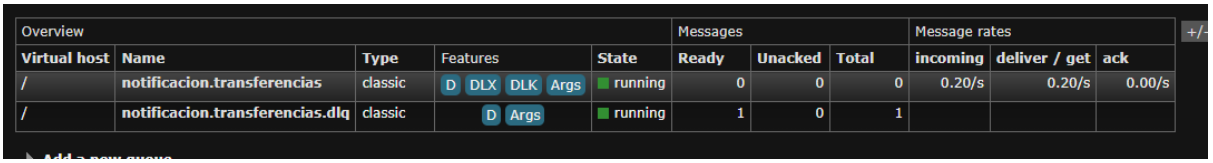
5. **La DLQ.**






| Overview | | | | | Messages | | | Message rates | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Virtual host | Name | Type | Features | State | Ready | Unacked | Total | incoming | deliver / get | ack |
| / | notificacion.transferencias | classic | D Args | running | 0 | 1 | 1 | 0.00/s | 0.00/s | 0.00/s |

Una vez enviado el mensaje envenenado de nuevo, ahora se va a la otra cola.



| Overview | | | | | Messages | | | Message rates | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Virtual host | Name | Type | Features | State | Ready | Unacked | Total | incoming | deliver / get | ack |
| / | notificacion.transferencias | classic | D DLX DLK Args | running | 0 | 0 | 0 | 0.20/s | 0.20/s | 0.00/s |
| / | notificacion.transferencias.dlq | classic | D Args | running | 1 | 0 | 1 | | | |

**¿Un JSON bien formado con un tipo que no existe, o sin eventoId, también es veneno?**

Sí, también es veneno. Aunque la sintaxis del JSON sea correcta, semánticamente es inprocesable porque el código no sabrá qué aviso generar o lanzará un NullPointerException, por lo que debe irse a la DLQ directamente.

**¿Y si lo que falló fue tu base de datos?**

En ese caso el mensaje NO es veneno, ya que su contenido está perfecto. La caída de la base de datos es un problema temporal de infraestructura.

6. **Preguntas de cierre.**
    a. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

Si RabbitMQ está caído justo cuando la saga llega a COMPLETADA, el intento de publicar el evento falla por falta de conexión y el aviso se pierde para siempre. Ocurre una "doble escritura" no transaccional: la base de datos local registra la transferencia como exitosa, pero el mensaje nunca sale hacia el broker. El mecanismo de publisher confirm no arregla este problema porque su función es que un broker vivo te devuelve un acuse de recibo; si RabbitMQ está completamente apagado, la conexión inicial fracasa y el mensaje nunca sale del servicio, utilizando cualquier espera de confirmación.

    b. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

El ack manual se debe enviar a RabbitMQ obligatoriamente después de hacer el commit de los avisos en la base de datos para asegurar una garantía de entrega al menos una vez. Si colocas el ack después y el servicio muere, se obtiene la garantía de que el aviso no se perderá: RabbitMQ asumirá que fallo y entregará el mensaje, permitiendo manejar el duplicado de forma segura mediante idempotencia. Por el contrario, si el orden fuera al revés (hacer ack antes de guardar) se obtendría una garantía de máximo una vez; si el servicio se apagará justo en ese momento intermedio, RabbitMQ borraría el evento creyéndolo procesado exitosamente y la notificación del cliente se perdería para siempre.

c. Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.

**Elección:** Se reemplazó la llamada HTTP síncrona al servicio de notificaciones por un modelo de publicación de eventos asíncronos mediante mensajería (RabbitMQ).

**Porque:** Era necesario desacoplar los sistemas para que la caída de un servicio secundario (las notificaciones) no tumbara una operación financiera prioritaria donde el dinero ya se había movido con éxito.

**Sacrificio:** Se sacrificó la consistencia estricta e inmediata de cara al cliente en favor de una consistencia eventual; ahora existe el riesgo de un desacople en tiempo donde el dinero llega a su destino, pero el usuario experimenta un retraso en recibir la confirmación.